Rearrange Linked List

AlgoExpert · Very Hard · Linked Lists

The problem

- Stable partition around k: nodes < k, then == k, then > k.
- Returns new head.

Intuition

- Build three separate lists for less, equal and greater, then join them.

Brute force vs optimal

- Copy values into arrays → O(n) space
- Three dummy lists → O(n), O(1) space

Approach

- 1. Walk the list appending to the right partition
- 2. Keep heads and tails
- 3. Connect the non-empty lists

Worked example



k = 3

Partition around k, keeping order

Worked example

< k



= k



> k



Three tail-appended lists

Worked example



Concatenate

Java solution

```java
public class RearrangeLinkedList {
    public static LinkedListNode rearrangeLinkedList(LinkedListNode head, int k) {
        LinkedListNode lo = new LinkedListNode(0), eq = new LinkedListNode(0), hi = new LinkedListNode(0);
        LinkedListNode l = lo, e = eq, h = hi;
        for (LinkedListNode c = head; c != null; c = c.next) {
            if (c.value < k) { l.next = c; l = c; }
            else if (c.value == k) { e.next = c; e = c; }
            else { h.next = c; h = c; }
        }
        h.next = null;
        e.next = hi.next;
        l.next = eq.next != null ? eq.next : hi.next;
        return lo.next;
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Edge cases & pitfalls

- Empty partitions
- Terminate the last list
- Stable order

Interview tips & follow-ups

- Mention stability

Recap

- Build three separate lists for less, equal and greater, then join them.
- Complexity: O(n)